

Fruit Ripeness Detection System

Submitted By

Sayma Mushsharat

Roll: 2007036

Submitted To

Dr. Sk. Md. Masudul Ahsan

Professor

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Md Tajmilur Rahman

Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology



Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna-9203, Bangladesh

November 5, 2025

Objectives

The objective of this project is to develop a system for detecting the ripeness stage of fruits using classical computer vision and image processing techniques. Specifically, the goals include:

- **Ripeness Classification:** To accurately classify fruit ripeness into three categories: Unripe, Ripe, and Overripe, based on visual cues such as color and texture.
- **User Interface:** Providing an intuitive and interactive interface for users to upload fruit images and view the analysis results, including classification and visualizations.
- **Scalability:** Ensuring the system can handle fruits of different types, sizes, and orientations, with consistent accuracy across varied conditions.
- **Real-time Visualization:** Displaying all intermediate processing steps in real-time, including color space conversion, edge detection, segmentation, and texture analysis.

Introduction

Fruit ripeness detection has gained significant attention due to its potential applications in agriculture, food processing, and quality control. This project focuses on recognizing the ripeness stage of fruits, classifying them into three categories: Unripe, Ripe, and Overripe. The system aims to identify the ripeness of fruits through the analysis of visual features, including color and texture.

Ripeness detection from input fruit images in this project involves detecting and classifying the fruit's ripeness using image processing techniques. The process includes preprocessing the image, segmenting the fruit from the background, and classifying the ripeness stage based on the fruit's color and texture. The methodology utilizes HSV (Hue, Saturation, Value) color space for color-based segmentation and applies texture analysis to identify surface irregularities indicative of overripeness.

This project uses OpenCV for image processing and Tkinter for the user interface.

Tools Used

- **Python:** The programming language used for development.
- **OpenCV:** For image processing tasks such as color space conversion, contour detection, segmentation, and morphological operations.
- **NumPy:** For numerical operations and matrix calculations.
- **Tkinter:** For creating the graphical user interface.
- **PIL (Python Imaging Library) / Pillow:** For handling and processing images.
- **Matplotlib:** For visualizing intermediate processing steps, such as displaying masks and segmentation results.

Methodology

Pipeline Overview

The Fruit Ripeness Detection system follows a multi-stage pipeline:

- **Image Preprocessing:** Convert to HSV color space and apply Gaussian blur
- **Fruit Segmentation:** Isolate fruit from background using edge detection and contours
- **Color-Based Segmentation:** Classify pixels as unripe, ripe, or overripe based on HSV ranges
- **Texture Analysis:** Calculate local variance to detect spots and wrinkles
- **Final Classification:** Combine both results and then classify the fruits.
- **Visualization:** Display all intermediate steps and final result in GUI

Each phase involves specific algorithms that contribute to the final classification of the fruit's ripeness.

User Interface

The User Interface (UI) is designed to be simple and intuitive with the help of tkinter framework, providing the user with the following functionalities:

- **Image Upload:** Users can upload fruit images through a dialog box.
- **Ripeness Analysis:** Once the image is uploaded, users can trigger the ripeness analysis.
- **Real-Time Visualization:** Users can view intermediate processing steps, including segmentation, color classification, and texture analysis.
- **Classification Results:** After processing, the system displays the classification results (Unripe, Ripe, Overripe) along with visual feedback.

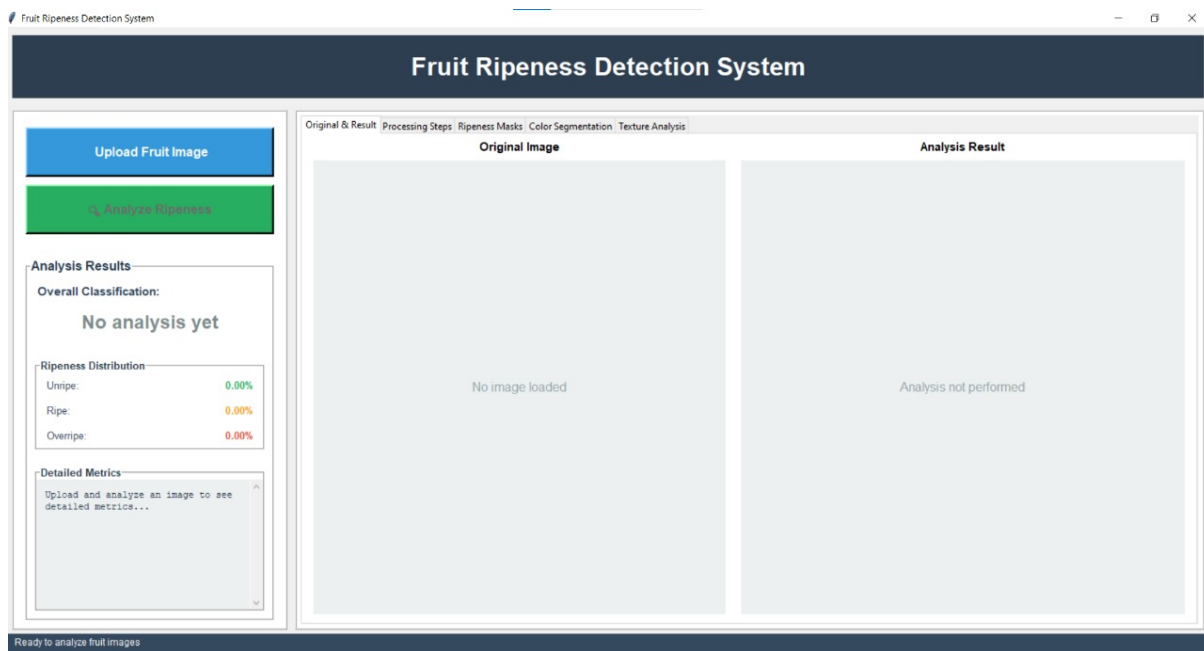


Figure 1: User Interface

Image Preprocessing

The first step is the Image Upload and Preprocessing phase. After the user uploads an image, it undergoes the following operations:

- **Color Space Conversion:** The image is converted from BGR (default color space) to HSV (Hue, Saturation, Value) for better color segmentation.
- **Gaussian Blur:** A Gaussian blur is applied to the image to reduce noise and make edge detection more accurate.

Algorithms Used:

- **HSV Color Space Conversion:** This conversion helps separate color information from brightness, making it easier to detect fruit colors.
- **Gaussian Blur:** The Gaussian blur smooths the image, reducing high-frequency noise and making edge detection more reliable.

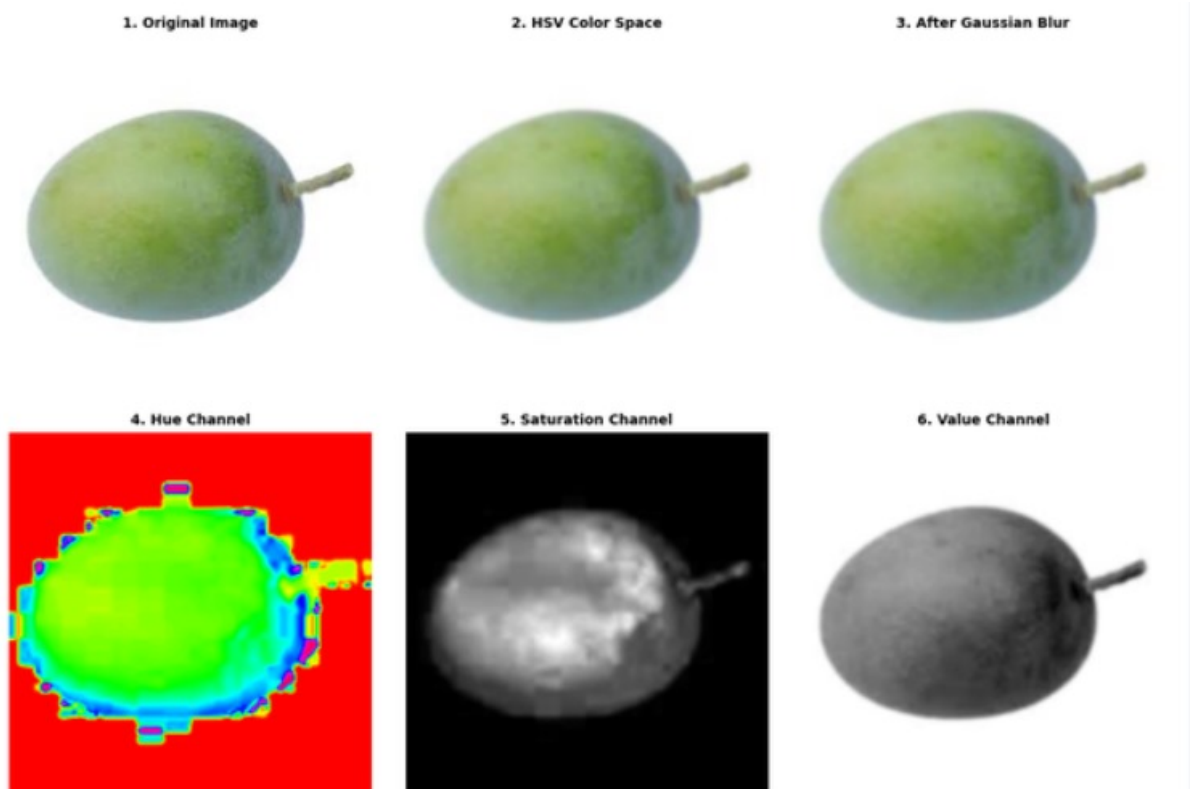


Figure 2: Image Preprocessing: Color Space Conversion and Gaussian Blur.

Fruit Segmentation

In the Fruit Segmentation phase, the system isolates the fruit from the background. This is achieved using the following techniques:

- **Edge Detection:** The Canny edge detection algorithm is applied to identify the boundaries of the fruit. This step highlights the sharp intensity changes around the fruit, making it easier to isolate it from the background.
- **Contour Detection and Morphological Operations:** After detecting the edges, the contours of the fruit are extracted. Morphological operations such as dilation and erosion are then applied to fill gaps in the detected contours and refine the fruit segmentation.

Algorithms Used:

- **Canny Edge Detection:** This algorithm detects edges by identifying areas of rapid intensity change in the image, effectively outlining the fruit's boundary. It works by applying a series of filters to detect gradients and then performing non-maximum suppression to refine the edges.
- **Contour Detection and Dilation:** After edge detection, contours are used to detect the fruit's shape and isolate it from the background. The dilation operation enhances the contours by filling gaps between segmented regions, improving the accuracy of the fruit mask.

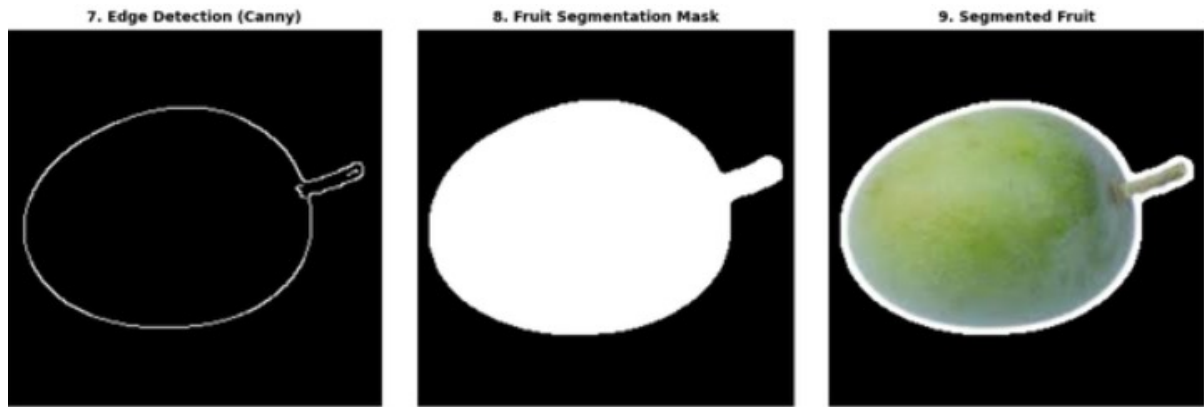


Figure 3: Fruit Segmentation

Ripeness Mask Generation

In the Ripeness Mask Generation phase, the system generates binary masks that represent different ripeness stages of the fruit (Unripe, Ripe, and Overripe). These masks are created based on color thresholds applied to the Hue, Saturation, and Value (HSV) channels. The thresholds are carefully selected to differentiate between unripe, ripe, and overripe fruits.

- **Unripe Mask:** The mask for unripe fruits is generated by detecting green hues (between 40° and 85° in Hue), with low saturation and high brightness (value ≥ 80). These conditions are typical for unripe fruits that are still green in color.
- **Ripe Mask:** The ripe fruits are identified by detecting hues in the red (0° to 10°) or yellow (15° to 50°) ranges, with high saturation and brightness. These fruits are often ripe and fully matured.
- **Overripe Mask:** Overripe fruits are detected by identifying dark hues (5° to 35° in Hue) and low brightness (value ≤ 100), which is characteristic of fruits that are decaying or past their prime.

The masks are generated using the `cv2.inRange()` function, which checks if each pixel's value falls within the specified range for each ripeness stage. The system then combines these masks to produce the final ripeness classification for each fruit.

Algorithms Used:

- **HSV Thresholding for Unripe Mask:**
 - The system uses the Hue channel to detect the green color typical of unripe fruits (between 40° and 85°).
 - The Saturation and Value channels are used to further restrict the mask to only those pixels that are saturated and bright enough to represent a green fruit.
 - The `cv2.inRange()` function is applied to generate the unripe mask.
- **HSV Thresholding for Ripe Mask:**

- The ripe fruits are identified by their red and yellow hues. The Hue channel ranges for red (0° to 10° and 170° to 180°) and yellow (15° to 50°) are used.
- High Saturation (70-255) and Value (100-255) ensure that only fully ripe fruits are selected.
- The `cv2.inRange()` function is used to isolate the pixels corresponding to ripe fruits.

- **HSV Thresholding for Overripe Mask:**

- Overripe fruits are identified by their brownish hues (5° to 35°) and low brightness (value ≤ 100).
- The Overripe mask is created by combining the detected dark regions (value ≤ 100) with the brownish hues (5° to 35° in Hue) and moderate saturation (40-255).
- The `cv2.inRange()` function is again used to apply these thresholds and isolate overripe fruit regions.

Once these individual masks (Unripe, Ripe, and Overripe) are generated, the system refines the masks using morphological operations to remove small noise and fill in gaps in the detected regions.

Morphological Operations:

- **Dilation:** The dilation operation is applied to the generated masks to ensure that the detected fruit region is fully enclosed and no gaps remain. It also connects broken parts of the fruit region.
- **Erosion:** After dilation, erosion is applied to remove small noisy regions that might have been incorrectly classified as part of the fruit.

These morphological operations help in creating smooth, accurate masks for each ripeness category.

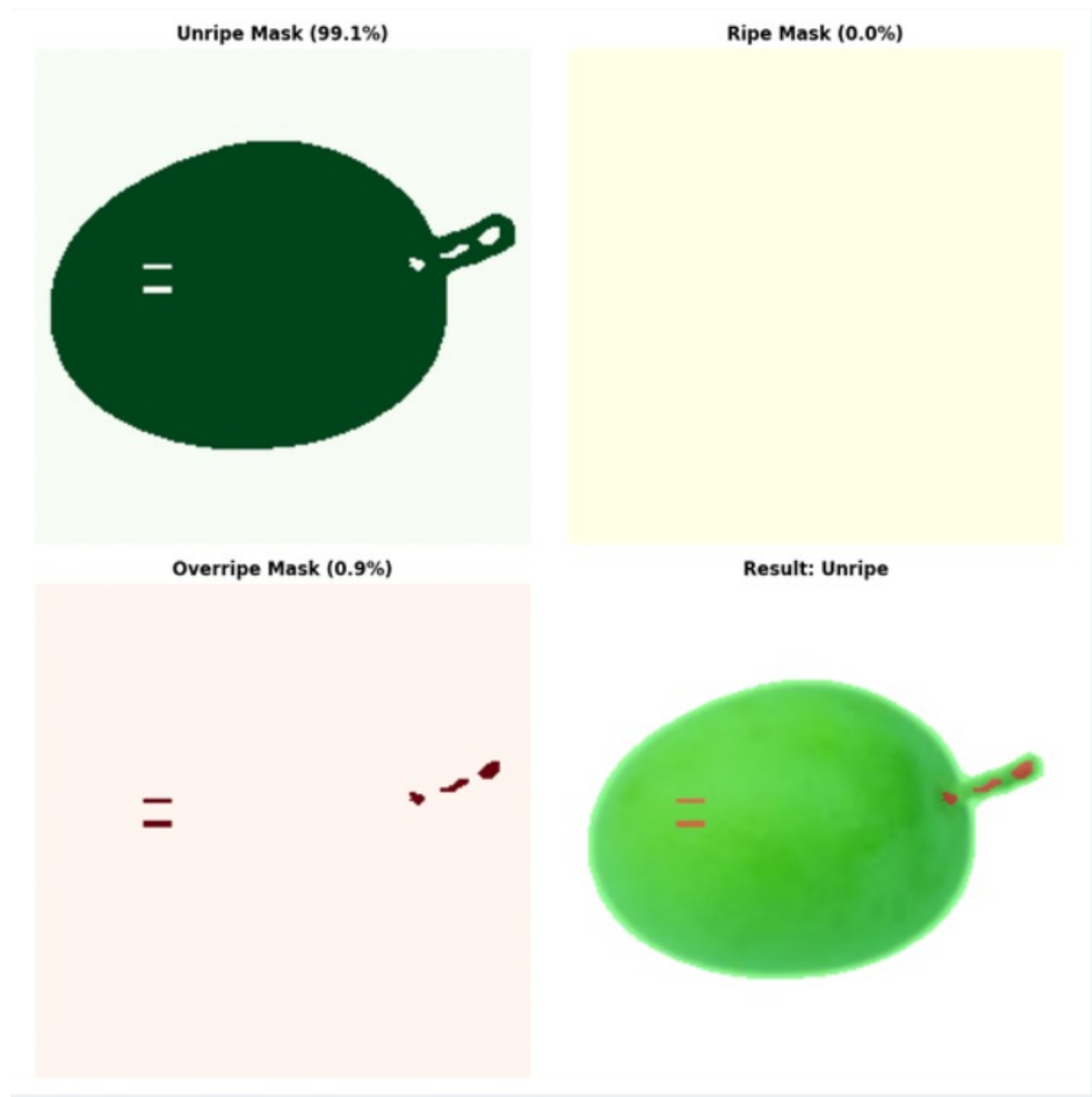


Figure 4: Ripeness masks generated for the fruit (Unripe, Ripe, Overripe)

Color Segmentation for Ripeness Classification

The Color Segmentation phase classifies the fruit into Unripe, Ripe, and Overripe categories based on color information in the HSV color space:

- **Unripe:** Green, low saturation, high brightness.
- **Ripe:** Red or yellow, high saturation and brightness.
- **Overripe:** Brown or dark, low brightness.

Algorithms Used:

- **HSV Thresholding:** Color ranges in the Hue, Saturation, and Value channels are used to identify Unripe, Ripe, and Overripe fruits.

- **Morphological Operations:** Dilation and erosion are used to refine the binary masks for better accuracy.

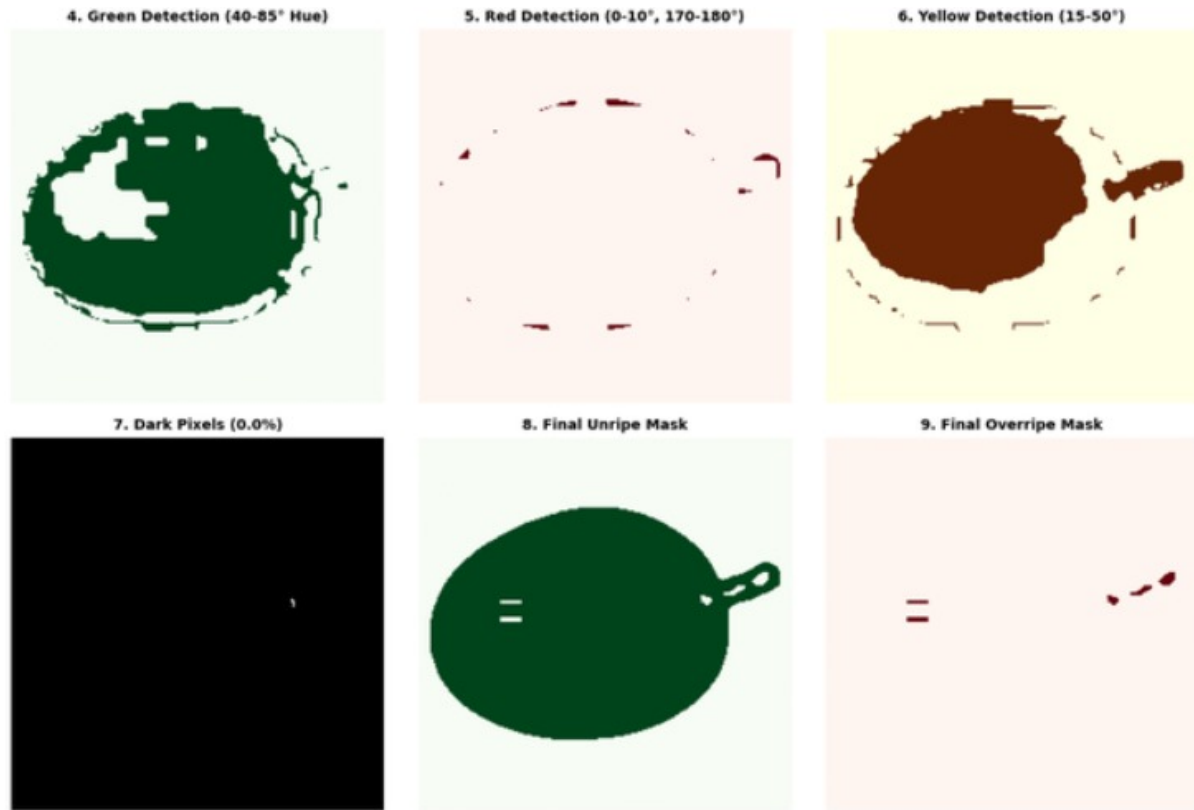


Figure 5: Color segmentation results showing Unripe, Ripe, and Overripe categories.

Texture Analysis

The Texture Analysis phase helps in detecting overripeness by examining the surface texture of the fruit. Fruits that are overripe typically exhibit more irregularities in texture, such as spots, wrinkles, or rough areas, which can be detected through pixel intensity variance.

Algorithms Used:

- **Local Variance Calculation:** The system calculates the variance of pixel intensities within a small window (usually 15×15 pixels) of the fruit mask. Local variance refers to the variability of pixel values in a localized region. High variance in a region generally indicates that the surface is irregular or rough, which is typical for overripe fruit. The algorithm is applied in the following manner:
 - The grayscale image of the fruit is used to compute the variance map.
 - A kernel of size 15×15 is applied to the image to calculate the local mean and mean of squared pixel values.
 - The variance is computed as the difference between the squared mean and the mean of the squared intensities.

- The result is a variance map that highlights areas with high variance, typically corresponding to rough or irregular surfaces of overripe fruit.
- **Texture Refinement:** After calculating the variance map, the system performs texture refinement by looking for areas where the local variance exceeds a threshold. These high-variance areas are then classified as overripe. Additionally, areas with lower variance are likely to be classified as ripe or unripe, as they show smoother textures. This process refines the fruit's ripeness classification by considering the texture characteristics of the surface.
- **Thresholding and Refining:** A threshold is applied to the calculated variance map to identify regions with sufficiently high variance that could indicate an overripe fruit. If the average variance in the fruit mask exceeds a certain value (e.g., 2000), it is classified as overripe. This threshold can be fine-tuned based on the texture characteristics of different fruits.

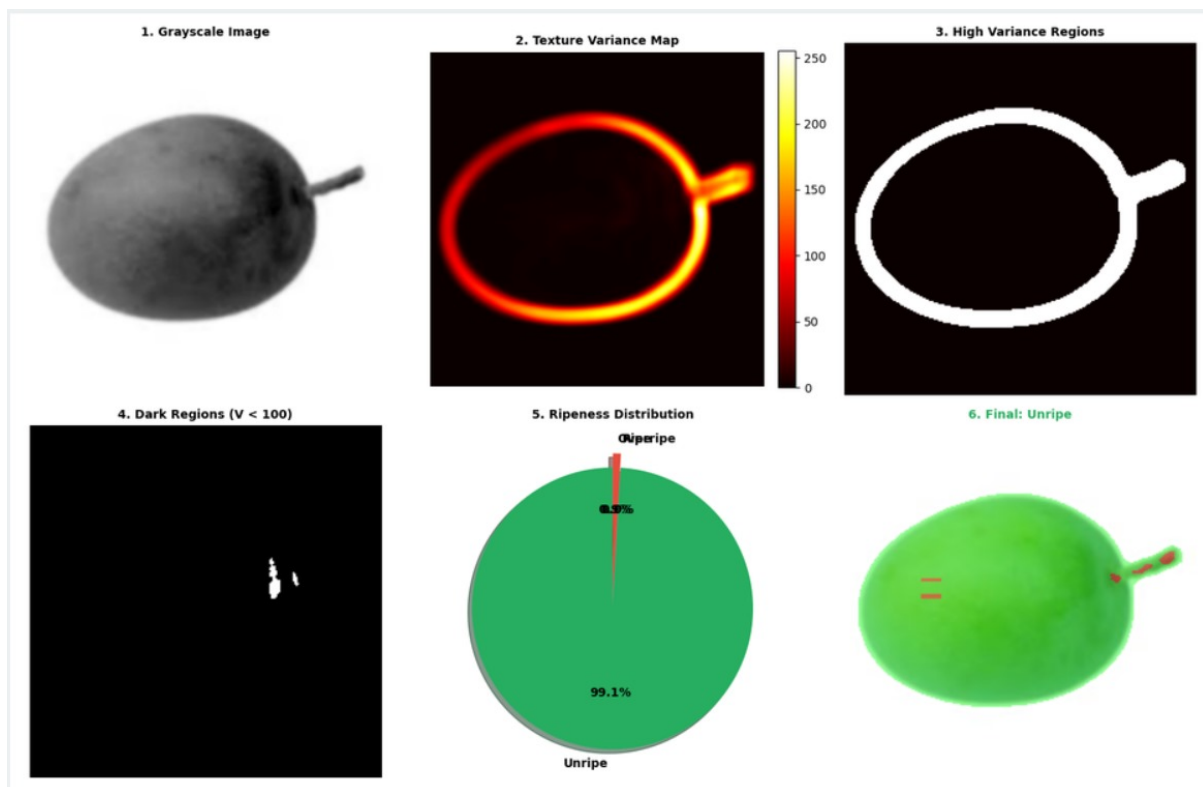


Figure 6: Texture analysis

Analysis Result Display

The User Interface shows the original image and the corresponding ripeness analysis result. The system classifies fruits with the analysis displaying the ripeness distribution and detailed texture analysis metrics.

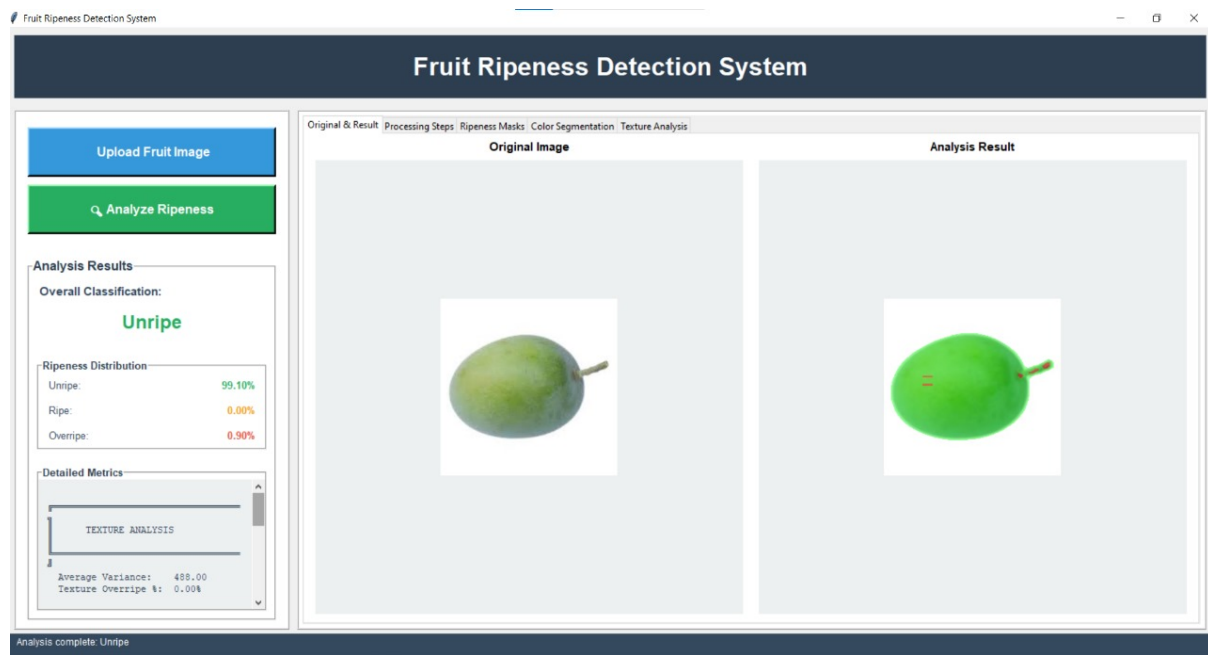


Figure 7: Final Result Analysis

Achievements

- **Robust Segmentation:** Successfully isolates fruits from complex backgrounds using edge detection and color thresholding.
- **Multi-Criteria Classification:** Combines color analysis and texture analysis for accurate ripeness detection.
- **Real-time Processing:** Processes typical fruit images in 3-5 seconds with responsive UI.
- **No Machine Learning Required:** Achieves good accuracy using only classical computer vision techniques.
- **Scalable Design:** Handles fruits of different sizes and orientations through normalization.
- **User-Friendly Interface:** Intuitive GUI with lazy loading for smooth user experience.

Limitations and Challenges

- **Lighting Sensitivity:** System performance degrades under non-uniform or extreme lighting conditions.
- **Background Complexity:** Complex backgrounds may interfere with fruit segmentation despite fallback mechanisms.
- **Fruit Type Specificity:** HSV ranges are calibrated primarily for bananas and may need adjustment for other fruits.
- **Occlusion Handling:** Partially occluded fruits may not be properly segmented.

- **Multiple Fruits:** Currently processes only one fruit per image (selects largest contour).
- **Color Variations:** Some fruit varieties have unusual color patterns that may confuse the classifier.

Discussion

The Fruit Ripeness Detection System utilizes classical image processing techniques to effectively classify the ripeness stages of fruits into three categories: Unripe, Ripe, and Overripe. The system follows a structured pipeline involving several stages, such as image preprocessing, segmentation, color-based classification, texture analysis, and final classification.

The core of the classification relies on color thresholds in the HSV color space, which provides better segmentation accuracy than RGB due to its separation of chromatic and brightness information. The Canny edge detection algorithm and contour-based segmentation help isolate the fruit from the background, making it easier to apply the classification logic.

In addition to color-based classification, texture analysis plays a vital role in determining the ripeness of the fruit. By calculating the local variance of the fruit's surface, the system detects rough textures, which are indicative of overripeness. The decision tree at the final stage combines both color and texture metrics to determine the overall classification.

However, the system's performance can be influenced by various factors. For instance, lighting conditions, fruit type variability, and background complexity may affect the accuracy of segmentation and classification. Moreover, the system currently processes one fruit at a time, and its accuracy could be reduced if the fruit is partially occluded or if the fruit color and texture do not match the predefined ranges for classification.

Conclusion

The Fruit Ripeness Detection System demonstrates the potential of classical computer vision techniques in automating the detection of fruit ripeness without relying on machine learning. The combination of color-based classification, edge detection, and texture analysis allows for robust and efficient ripeness classification. Despite some limitations, such as sensitivity to lighting and fruit variety, the system provides reliable results in most cases and can be extended with future enhancements, such as adaptive algorithms, multi-fruit processing, and integration with mobile platforms for real-time applications in agriculture, food processing, and quality control.

References

- [1] Gonzalez, R. C., Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson Education. ISBN: 978-0133356724.
- [2] Bradski, G., Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media. ISBN: 978-0596516130.

-
- [3] Course Materials. CSE 4128: Image Processing and Computer Vision Laboratory.